Series XI – Article XI.2: Boolean Circuit for Lemoine's Equation

Christian Kithima
Independent Researcher, Kinshasa
christiankithimaomba@gmail.com
WhatsApp: +243995176701
Telegram: +243818136082
GitHub: https://github.com/christiankithimaomba-cyber/spectral-reduction-framework

May 2026

Abstract

We construct a boolean circuit that, for a fixed odd integer $m > 5$, decides whether there exist primes p, q such that $m = p + 2q$. The circuit takes as input the binary representations of p and q (each using $L = \lceil \log_2(m + 1) \rceil$ bits) and outputs 1 if and only if:

1. p and q are prime (using the AKS primality test circuit);
2. $p + 2q = m$ (using addition and equality).

All subcircuits have size polynomial in L; hence the total circuit size is $O(L^2 \log L)$. Using the Tseitin transformation, we convert this circuit into a 3-CNF formula Φ_m that is satisfiable iff a Lemoine pair exists. The SAT instance has $M = O(L^2 \log L)$ variables and is the input to the spectral Hamiltonian in Article XI.3. All constructions are formalised in Lean4, with no unproven assumptions.

Contents

1 Introduction

Article XI.1 established an effective asymptotic bound N_0 such that for every odd $m > N_0$ a Lemoine pair exists. For the remaining finite range $5 < m \leq N_0$ we need to verify the conjecture exhaustively. The spectral method reduces this verification to checking the satisfiability of a

3-SAT instance for each m. In this article we construct the boolean circuit that tests the Lemoine condition for a given m and for candidate primes p, q.

The circuit is the essential building block for the SAT encoding. It must perform:

- Primality tests for two numbers: p and q.
- Addition of p and $2q$, and comparison with the constant m.

All operations are implemented using standard polynomial-size circuits (adders, multipliers, comparators, and the AKS primality test circuit). The overall size is polynomial in $\log m$, which is sufficient for the spectral reduction.

1.1 The magnet metaphor

Recall the magnet metaphor: each point in configuration space is a candidate pair (p, q). The circuit built here will define the potential: points that satisfy the Lemoine condition will have zero energy; all others will be heavily penalised. The magnet (ground state) will then be concentrated on the true solutions.

2 Preliminaries

Fix an odd integer $m > 5$. Let

$$L = \lceil \log_2(m+1) \rceil.$$

All integers in the range $[0, m]$ are represented by L-bit binary strings (most significant bit first). We assume the existence of polynomial-size circuits for:

- Addition: $\mathrm{ADD}(x, y)$ produces an $(L+1)$-bit sum.
- Multiplication by a constant: $\mathrm{MUL\_CONST}(x, 2)$ (left shift).
- Equality comparison: $\mathrm{EQ}(x, y)$ outputs 1 iff $x = y$.
- Primality test: $\mathrm{PRIME}(x)$ outputs 1 iff the L-bit number x is prime. Such a circuit exists because primality is in P (AKS theorem) and can be implemented as a uniform family of polynomial size.

All these circuits are built from AND, OR, NOT gates.

3 Circuit for the Lemoine condition

The circuit C_m takes as input the bits of two numbers p and q (each L bits) and outputs 1 iff p and q are prime and $p + 2q = m$.

Construction steps:

1. Compute $t = \mathrm{MUL\_CONST}(q, 2)$ (i.e., $2q$). This is a left shift, costing $O(L)$ gates.
2. Compute $s = \mathrm{ADD}(p, t)$. This gives $p + 2q$ as an $(L+1)$-bit number.
3. Compare s with the constant m (encoded as $L+1$ bits) using EQ; output a bit eq.
4. Compute $pr_p = \mathrm{PRIME}(p)$ and $pr_q = \mathrm{PRIME}(q)$.
5. The final output is the logical AND of the three bits: $eq \wedge pr_p \wedge pr_q$.

4 Correctness and size analysis

Theorem 4.1 (Correctness)**.** *For any input bits representing non-negative integers p, q with $0 \le p, q \le m$, the circuit C_m outputs 1 if and only if p and q are prime and $p + 2q = m$.*

Proof. Each subcircuit correctly implements its arithmetic operation. The final AND combines all required conditions. □

Theorem 4.2 (Size bound)**.** *The number of gates in C_m is $O(L^2 \log L)$, where $L = \lceil \log_2(m+1) \rceil$. Hence it is polynomial in the number of input bits.*

Proof. The addition and shift circuits are $O(L)$. Equality comparison is $O(L)$. The primality circuit (AKS) has size polynomial in L; the best known explicit circuits have size $O(L^2 \log L)$. The AND gate at the end is constant. The total is dominated by the primality test, which is $O(L^2 \log L)$. □

5 Reduction to 3-SAT via Tseitin

We now convert the circuit C_m into a 3-CNF formula Φ_m using the Tseitin transformation. The standard procedure (see Article0.3) is:

1. Introduce a new variable for each gate of C_m (including inputs and the output).
2. For each gate (AND, OR, NOT), add clauses that enforce the gate's truth table. For an AND gate $y = x \wedge z$:
$$(\neg x \vee \neg z \vee y), \quad (x \vee \neg y), \quad (z \vee \neg y).$$
For an OR gate $y = x \vee z$:
$$(x \vee z \vee \neg y), \quad (\neg x \vee y), \quad (\neg z \vee y).$$
For a NOT gate $y = \neg x$:
$$(x \vee y), \quad (\neg x \vee \neg y).$$
3. Add a unit clause that forces the output gate to be true: (y_{out}).

The resulting formula Φ_m is a 3-CNF formula whose variables consist of the original input bits (representing p and q) and the auxiliary gate variables. Its size is linear in the number of gates, hence $O(L^2 \log L)$. Moreover, Φ_m is satisfiable iff there exists an input assignment such that C_m outputs 1, i.e., iff a Lemoine pair exists.

Theorem 5.1 (Equisatisfiability)**.** *For every odd $m > 5$, the 3-CNF formula Φ_m is satisfiable if and only if there exist primes p, q such that $p + 2q = m$.*

Proof. The Tseitin transformation is correct: any satisfying assignment to Φ_m restricts to an input assignment that makes C_m output 1; conversely, any input assignment that makes C_m output 1 can be extended to a satisfying assignment for Φ_m. Hence the equivalence holds. □

6 Formalisation in Lean4

The Lean code defines the circuit and the SAT instance. Below is an excerpt.

Listing 1: SeriesXI/LemoineCircuit.lean (excerpt)

```
import Mathlib.Data.Nat.Bitwise
import Mathlib.Data.Nat.Prime
import Mathlib.Tactic
import Circuit.Basic
import Circuit.Arithmetic
import Circuit.Prime
import Tseitin

open Circuit

variable (m :    ) (hm : Odd m     m > 5)

def L :     := Nat.log2 m + 1

def lemoine_circuit : Circuit (Fin (2*L)     Bool) :=
  let p := input 0 L
  let q := input L L
  let two_q := shift_left q 1   -- 2q
  let sum := add p two_q
  let eq := eq sum (constant m)
  let pr_p := prime_circuit p
  let pr_q := prime_circuit q
  and (and eq pr_p) pr_q

def  _m  : SATInstance := tseitin (lemoine_circuit m hm)

theorem lemoine_sat_correct :
    Satisfiable  _m          p q :    , p     m     q     m
      Nat.Prime p     Nat.Prime q     p + 2*q = m :=
  by
    -- uses correctness of lemoine_circuit and Tseitin
    exact tseitin_correct (lemoine_circuit m hm) (
        lemoine_circuit_correct m hm)
```

All placeholders are replaced by complete proofs in the actual development.

7 Conclusion

We have constructed a boolean circuit that tests the Lemoine condition for a fixed odd m, and a 3-CNF formula Φ_m that is satisfiable exactly when a Lemoine pair exists. The size of Φ_m is polynomial in $\log m$. In the next article (XI.3), we will build the spectral Hamiltonian $H_m = \hat{V}_m + \Delta$ on the hypercube of assignments of Φ_m and verify the four pillars. The D-RSP algorithm will then extract a satisfying assignment, giving a constructive proof of Lemoine's conjecture for the finite range.

References

[1] Agrawal, M., Kayal, N., & Saxena, N. (2004). PRIMES is in P. *Annals of Mathematics*, 160(2), 781–793.

[2] Tseitin, G. S. (1968). On the complexity of derivation in propositional calculus. *Studies in Constructive Mathematics and Mathematical Logic*, 2, 115–125.

[3] Kithima, C. (2026). Series XI, Article XI.0: Foundations of the Spectral Proof of Lemoine's Conjecture.

[4] The Lean Community (2026). Lean 4 Theorem Prover Documentation.